DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF COMPUTING & ARTIFICIAL INTELLIGENCE

# PROJECT DOCUMENT

*Data Structures And Algorithms*

## BS(CS)-3B

## AIR UNIVERSITY ISLAMABAD

## Group Members

| Name | Registration-ID |
|---|---|
| Zumer Dhillun | 231597 |
| Duaa Dara | 232481 |
| Zoya Azad | 231579 |
| Afia Aziz | 231561 |

## Submitted to: Sir Wahab Ali

# Rail Ease: A Railway Reservation System

*Technical Manual*

**Table of Contents**

**Abstract**

Rail Ease is a command-line railway reservation system designed as a Data Structures and Algorithms course project. It demonstrates the practical application of linked lists, queues, maps, and graph algorithms. With no GUI, the system focuses on backend efficiency and functionality, enabling users and administrators to manage reservations, view train schedules, and optimize routes.

**Introduction**

**Purpose:** Rail Ease provides a platform for managing railway reservations efficiently while showcasing the implementation of various data structures.

**Project Significance:**

- **Educational Value:** Highlights core DSA concepts such as linked lists, queues, and graphs.
- **Efficiency:** Optimized route calculations using shortest path algorithms.
- **Scalability:** Designed for command-line systems with the potential for future GUI expansion.

**System Requirements**

**Hardware Requirements:**

- Processor: Intel Core i5 or higher
- RAM: 8 GB minimum
- Storage: 500 MB free space

**Software Requirements:**

- Operating System: Windows 10 or later
- Compiler: GCC or MinGW

**Dependencies:**

- Standard C++ libraries
- STL for containers
- iostream, string, and algorithm libraries

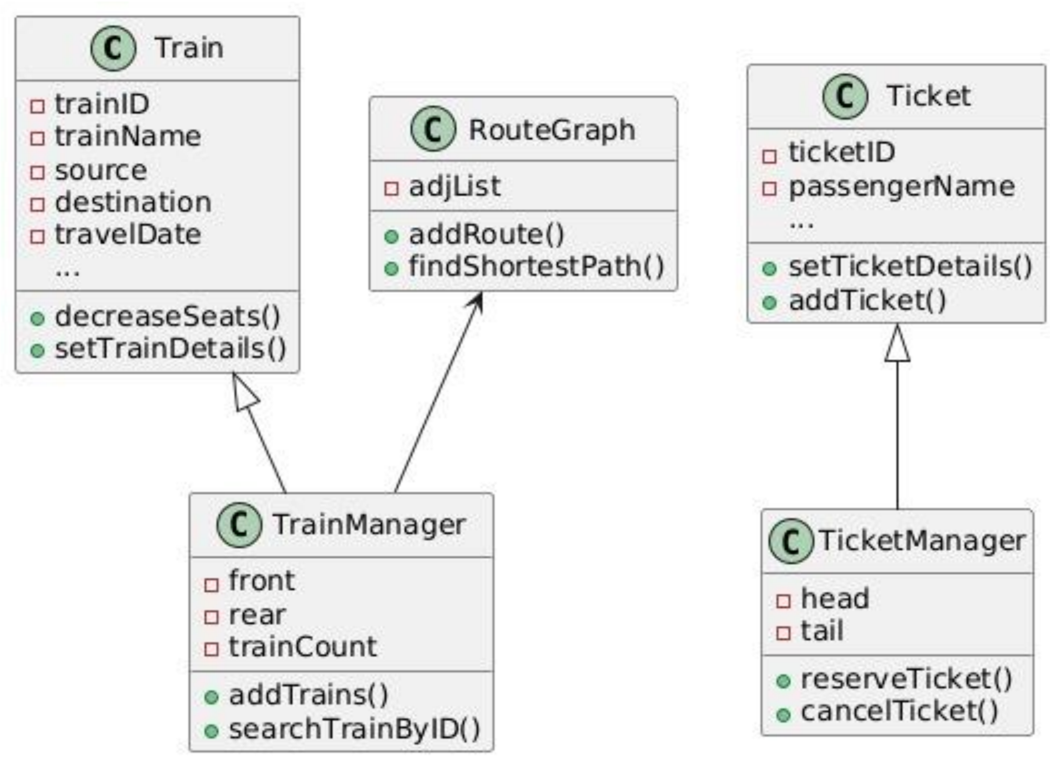
**System Design**

**Architecture Overview:** Rail Ease comprises modules for user interaction, ticket management, train management, and route optimization. It utilizes a linked list for train records, maps for adjacency lists, and queues for route traversal.

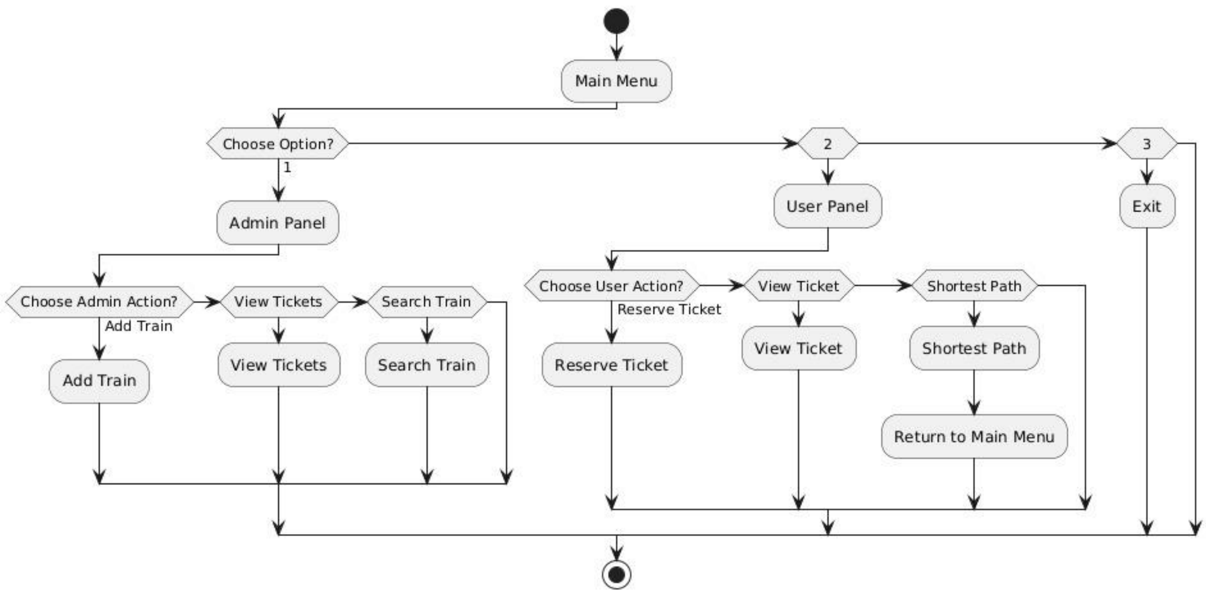
**Data Structures Implementation:**

- **Linked List:** Used for managing trains and tickets.
- **Queue:** Facilitates departure operations.
- **Map:** Implements the graph for route management.

**UML Class Diagram:**




**Flow Chart:**




**Implementation Details**

**Key Components:**

1. **Admin Panel:**
   - Manage trains, add trains, depart trains, search train, and view route-specific trains, display train records.
   - Credential validation implemented.

***Code Snippet: Credential Validation***

```
bool validateCredentials(string user, string password)
{
    return (user == "AfiaAziz" && password == "Afia231561") || (user == "ZumerDhillun" && password == "Zumer231597")
    || (user == "ZoyaAzad" && password == "Zoya231579") || (user == "DuaaDara" && password == "Duaa232481");
}
```

***Add Trains***

```cpp
void addTrains()
{
    string choice;
    double distance; // Declare the distance variable as double
    do
    {
        Train *newTrain = new Train();
        string input;
        int seats;

        cin.ignore(); // Clear leftover input buffer at the start of the loop
        cout << "\t\t\t\t\t\t\tEnter Train Information:" << endl;

        cout << "\t\t\t\t\t\t\tTrain ID: ";
        getline(cin, input);
        newTrain->setTrainID(input);

        cout << "\t\t\t\t\t\t\tTrain Name: ";
        getline(cin, input);
        newTrain->setTrainName(input);
```

### *Depart Trains*

```cpp
void departTrain()
{
    if (isEmpty())
    {
        cout << "\n\t\t\t\t\t\t\tNo trains available to depart." << endl;
        return;
    }

    // Take the train at the front of the queue
    current = front;
```

### *Search Train*

```cpp
// Search for a train by its ID
Tabnine | Edit | Test | Explain | Document | Ask
void searchTrainByID(string trainID)
{
    if (isEmpty())
    {
        cout << "\n\t\t\t\t\t\t\tNo trains available to search.\n";
        return;
    }

    current = front; // Start from the front of the queue
```

### *View Train By route*

```
void displayTrainsByRoute()
{
    string departureCity, destinationCity;
    bool trainFound = false;
    cin.ignore();
    // Keep asking for input until trains are found or the user exits
    do
    {
        cout << "\n\t\t\t\t\t\t\tEnter Departure City: ";
        getline(cin, departureCity);

        cout << "\t\t\t\t\t\t\tEnter Destination City: ";
        getline(cin, destinationCity);

        if (isEmpty())
        {
            cout << "\n\t\t\t\t\t\t\tNo trains available to search by route.\n";
            return;
        }
```

### *Display Train Records*

```
void displayAllTrainRecords()
{
    if (isEmpty())
    {
        cout << "\n\t\t\t\t\t\t\tNo trains available to display.\n";
        return;
    }

    current = front;
    cout << "\n\t\t\t\t\t\t\tTrain List:" << endl;
    cout << "------------------------------------------------------------------------------------

    while (current != nullptr)
```

**User Panel:**

- Reserve tickets, view bookings, cancel tickets, edit details, and find shortest routes.

### *Code Snippets:*

### *Reserve a Ticket*

```
void reserveTicket(TrainManager *train)
{
    string fName, lName, NIC, Contact;
    int age;
    Ticket *newTicket = new Ticket(); // Create a new ticket instance

    // Get passenger details
    cout << "\n\t\t\t\t\t\t\tEnter First Name: ";
    cin >> fName;
```

### *View Your Reservation*

```
void viewTicket()
{
    int search;
    string choice;
    bool ticketFound = false;
    current = head;

    while (true)
    {
        cout << "\t\t\t\t\t\t\tDo You Want to View Your First Booking? Press 1\n";
        cout << "\t\t\t\t\t\t\tDo You Want to Search Other Bookings? Press 2\n";
        cout << "\t\t\t\t\t\t\tEnter Choice (1-2): ";
        cin >> choice;
        if (choice == "1")
        {
```

### *Display All Tickets*

```
void displayAllTickets()
{
    if (isEmpty())
    {
        cout << "\t\t\t\t\t\t\tList is empty, nothing to display.\n";
        return;
    }
    current = head;
    cout << "\n\t\t\t\t\t\t\tTicket List:\n";
    cout << "---------------------------------------------------------------------------------------
```

### *Find Shortest Path*

```
// Find the shortest path between stations
void findShortestPath(const string& start, const string& end) {
    routeGraph.findShortestPath(start, end);
}
```

### *Cancel a Ticket*

```
void cancelTicket(int ticketID)
{
    if (isEmpty())
    {
        cout << "\t\t\t\t\t\t\tNo tickets available to cancel.\n";
        return;
    }
```

### *Edit Ticket Details*

```cpp
void editDetails()
{
    string fName, lName, NIC, Contact;
    int search;

    cout << "\t\t\t\t\t\t\tEDIT DETAILS:\n\n";

    if (isEmpty())
    {
        cout << "\t\t\t\t\t\t\tNo tickets available to edit.\n";
        return;
    }
```

### *Seat Choose*

```cpp
string SeatChoose()
{
    cout << "\n\n\t\t\t\t\t\t\tDisplay the Sitting Arrangement" << endl;
    cout << "\n\n\t\t\t\t\t\t\tA" << "\t" << " B" << "\t" << " C" << "\t\n";
    for (int i = 0; i < rows; i++)
    {
        cout << "\n\n\t\t\t\t\t\t\t";
        for (int j = 0; j < cols; j++)
        {
            cout << Seats[i][j] << "\t";
        }
        cout << endl;
    }

    string SeatNum;
    bool found = false;
```

### Libraries and Frameworks

- **C++ STL:** For containers and algorithms.
- **Io-stream and string:** For input and output.

### Testing and Results

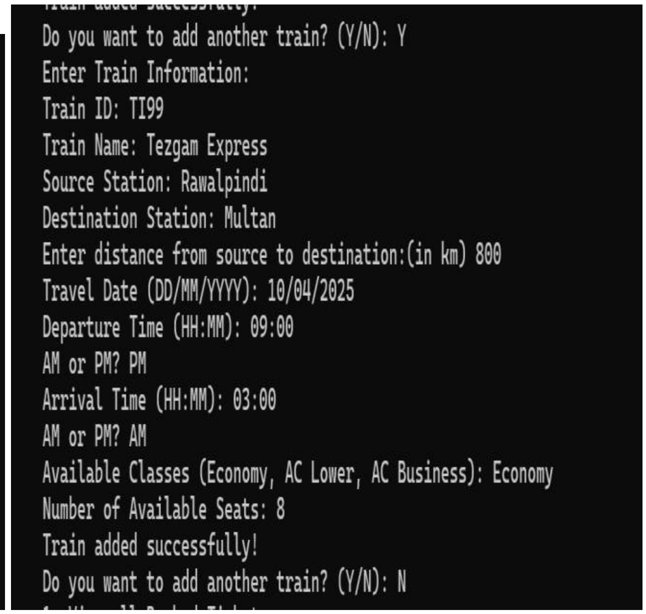
### Unit Tests:

- Verified individual methods for ticket reservation, train addition, and shortest path calculation.

### Performance Tests:

- Train addition: Average time ~0.05 seconds per train.
- Shortest path: Optimized using priority queues.

**Screenshots of the Output:**



```
******** Welcome to Train Ticket Management System **********************
            1. User Panel
            2. Admin Panel
            3. Exit
            Enter your choice: 1

*********************** User Pannel *************************************
            1. Reserve a Ticket
            2. View Your Reservation
            3. Display All Tickets
            4. Find Shortest Path
            5. Cancel a Ticket
            6. Edit Ticket Details
            7. Return to Main Menu
            Enter your choice: 1

            Enter First Name: Afia
            Enter Last Name: Aziz
            Enter Age: 19
            Enter CNIC Number: 3768909876554
            Enter Contact Number: 03358976543
            Choose Train Class (Economy, AC Lower, AC Business): AC Lower

            Ticket Class: AC Lower [Berth Included]
            Ticket Price: 3500 PKR


            Enter Departure City: Rawalpindi
            Enter Destination City: Multan

            Trains Available on Route from Rawalpindi to Multan:
```

```
=============== PAYMENT DETAILS ===============
Choose Payment Method:
1. JazzCash
2. EasyPaisa
3. Credit/Debit Card


Enter Choice (1-3): 1


Enter JazzCash Mobile Account Number: 03345678909
Enter Account Holder's First Name: Ali
Enter Account Holder's Last Name: Ahmed


Payment successful! Thank you for using our service.


Ticket Reserved Successfully! Your Ticket ID is: 20742
```

```
Updated Sitting Arrangement

A       B       C

1       11      XX

2       12      22

3       13      23

4       14      24

5       15      25

6       16      26

7       17      27

8       18      28

9       19      29

10      20      30
```

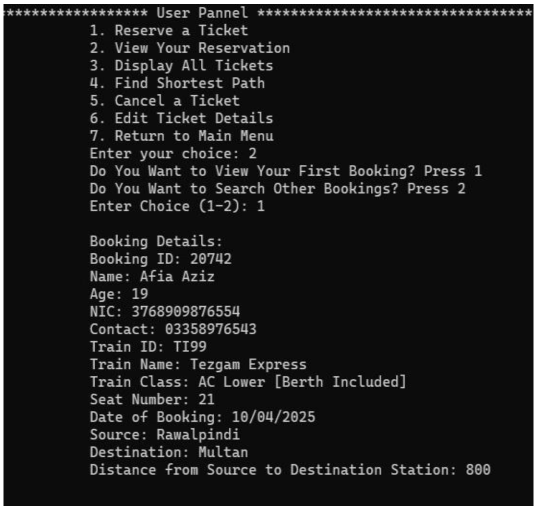
```
Display the Sitting Arrangement

A       B       C

1       11      21

2       12      22

3       13      23

4       14      24

5       15      25

6       16      26

7       17      27

8       18      28

9       19      29

10      20      30

Enter the seat number that you chose: 21
```

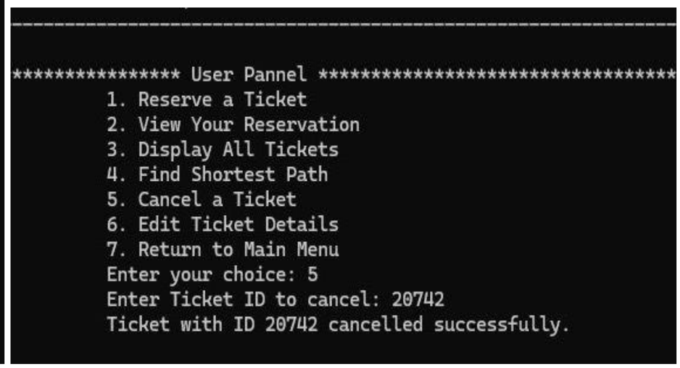
```
****************** User Pannel ******************************
          1. Reserve a Ticket
          2. View Your Reservation
          3. Display All Tickets
          4. Find Shortest Path
          5. Cancel a Ticket
          6. Edit Ticket Details
          7. Return to Main Menu
          Enter your choice: 6
          EDIT DETAILS:

          Enter ticket ID to edit: 20742
          Ticket found. Enter new details:
          Enter First Name of Passenger: Afia
          Enter Last Name of Passenger: Aziz
          Enter CNIC Number (13 digits): 3456789098776
          Enter Contact Number (11 digits): 03358907654
          Ticket details updated successfully.

****************** User Pannel ******************************
          1. Reserve a Ticket
          2. View Your Reservation
          3. Display All Tickets
          4. Find Shortest Path
          5. Cancel a Ticket
          6. Edit Ticket Details
          7. Return to Main Menu
          Enter your choice: 3

          Ticket List:
--------------------------------------------------------------
          Ticket ID: 20742
          Passenger Name: Afia Aziz
          Age: 19
          CNIC: 3456789098776
          Contact: 03358907654
          Train Name: Tezgam Express
          Train ID: 20742
          Source: Rawalpindi
          Destination: Multan
          Travel Date: 10/04/2025
          Seat Number: 21
          Price: $3500
--------------------------------------------------------------
```

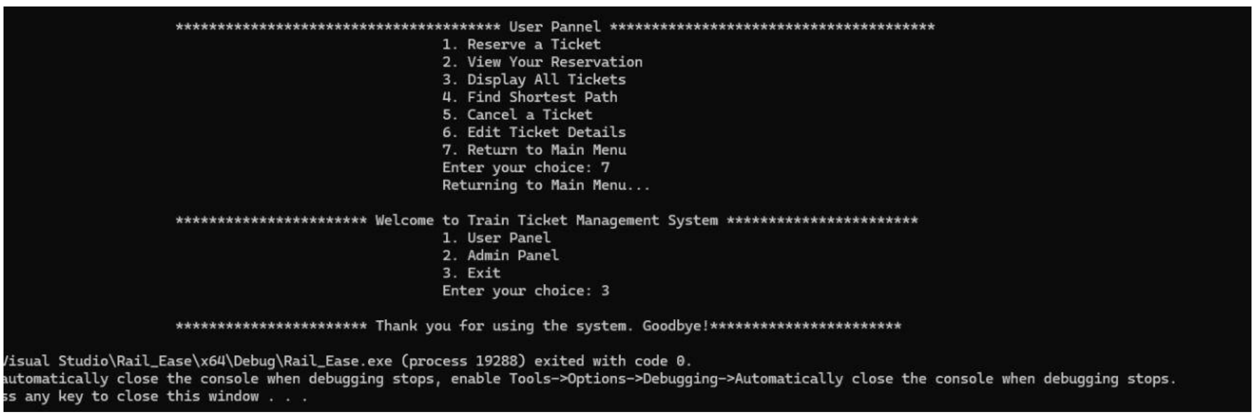
```
****************** User Pannel ****************
          1. Reserve a Ticket
          2. View Your Reservation
          3. Display All Tickets
          4. Find Shortest Path
          5. Cancel a Ticket
          6. Edit Ticket Details
          7. Return to Main Menu
          Enter your choice: 3

          Ticket List:
----------------------------------------------
          Ticket ID: 20742
          Passenger Name: Afia Aziz
          Age: 19
          CNIC: 3768909876554
          Contact: 03358976543
          Train Name: Tezgam Express
          Train ID: 20742
          Source: Rawalpindi
          Destination: Multan
          Travel Date: 10/04/2025
          Seat Number: 21
          Price: $3500
----------------------------------------------
```

```
*********** User Pannel ***********************************
   1. Reserve a Ticket
   2. View Your Reservation
   3. Display All Tickets
   4. Find Shortest Path
   5. Cancel a Ticket
   6. Edit Ticket Details
   7. Return to Main Menu
   Enter your choice: 2
   Do You Want to View Your First Booking? Press 1
   Do You Want to Search Other Bookings? Press 2
   Enter Choice (1-2): 2
   Enter Your Booking ID: 20742

   Booking Details:
   Booking ID: 20742
   Name: Afia Aziz
   Age: 19
   NIC: 3768909876554
   Contact: 03358976543
   Train ID: TI99
   Train Name: Tezgam Express
   Train Class: AC Lower [Berth Included]
   Seat Number: 21
   Date of Booking: 10/04/2025
   Source: Rawalpindi
   Destination: Multan
   Distance from Source to Destination Station: 800
```

```
************ User Pannel ***************************
    1. Reserve a Ticket
    2. View Your Reservation
    3. Display All Tickets
    4. Find Shortest Path
    5. Cancel a Ticket
    6. Edit Ticket Details
    7. Return to Main Menu
    Enter your choice: 4
    Enter starting station: Rawalpindi
    Enter destination station: Multan
    Shortest path: Rawalpindi -> Multan -> END
    Shortest Distance: 800 km
    Train ID: TI99
    Train Name: Tezgam Express
    Available Classes: Economy
    Seats Available: 8
```




**User Manual**

**Installation:**

1. Clone the repository.
2. Compile using `g++ -o RailEase RailEase.cpp`.
3. Run the program using `./RailEase`.

**Usage Guide:**

1. Select **User Panel** or **Admin Panel**.
2. Follow on-screen instructions for operations like booking or train management.

**Admin Manual**

1. Login with the following credentials:
   - Username: AfiaAziz, Password: Afia231561
   - Username: ZumerDhillun, Password: Zumer231597

- Username: ZoyaAzad, Password: Zoya231579
- Username: DuaaDara, Password: Duaa232481

### *Code Snippet: Admin Panel Options*

```cpp
void showAdminOptions()
{

    cout << "\t\t\t\t\t\t\t1. View all Booked Tickets\n";
    cout << "\t\t\t\t\t\t\t2. Add Trains\n";
    cout << "\t\t\t\t\t\t\t3. Depart Train\n";
    cout << "\t\t\t\t\t\t\t4. Search Train\n";
    cout << "\t\t\t\t\t\t\t5. View Train By route\n";
    cout << "\t\t\t\t\t\t\t6. Display Train Records\n";
    cout << "\t\t\t\t\t\t\t7. Return to Main Menu\n";
}
```

### *User Panel Options*

```cpp
    int choice;
    do
    {
        cout << "\n\t\t\t**************************************** User Pannel ************
        cout << "\t\t\t\t\t\t\t1. Reserve a Ticket\n";
        cout << "\t\t\t\t\t\t\t2. View Your Reservation\n";
        cout << "\t\t\t\t\t\t\t3. Display All Tickets\n";
        cout << "\t\t\t\t\t\t\t4. Find Shortest Path\n";
        cout << "\t\t\t\t\t\t\t5. Cancel a Ticket\n";
        cout << "\t\t\t\t\t\t\t6. Edit Ticket Details\n";
        cout << "\t\t\t\t\t\t\t7. Return to Main Menu\n";
        cout << "\t\t\t\t\t\t\tEnter your choice: ";
        cin >> choice;

        switch (choice)
```

**Test Cases**

1. **Add Train Test:**
    - Input valid details and ensure they are stored correctly.

### *Code Snippet: Train Addition*

```cpp
void addTrains() {
string choice;
double distance;  // Declare the distance variable as double
do {
    Train* newTrain = new Train();
    string input;
    int seats;
```

2. **Reserve Ticket Test:**
    - Ensure ticket details are stored and linked properly.

### *Code Snippet: Ticket Reservation*

```
void reserveTicket(TrainManager* train) {
    string fName, lName, NIC, Contact;
    int age;
    Ticket* newTicket = new Ticket();  // Create a new ticket instance
```

3. **Shortest Path Test:**
    - Input start and end stations; verify correct path output.

## Conclusion

Rail Ease successfully implements core data structures and algorithms to create an efficient railway reservation system. Future improvements could include a graphical user interface and database integration.

## References


1. C++ Reference Documentation: https://cplusplus.com/
2. Algorithms and Data Structures: Goodrich et al.
3. GeeksforGeeks Data Structures: https://www.geeksforgeeks.org/data-structures/
4. HackerRank Data Structures Track: https://www.hackerrank.com/domains/tutorials/10-days-of-dsa
5. Data Structures and Algorithms in C++ (Book): Mark Allen Weiss.